

Algoritmos y Estructuras de Datos.

- Clase 9 - Archivos -

Manejo de Archivos la libreria "paths"

Ya hemos visto como incluir librerias y archivos `.py`, por ejemplo cuando se encuentran en el mismo directorio. También es posible leer archivos `.py` que se encuentren en otros directorios, además de poder abrir los contenidos de distintos tipos de archivos (i.e. txt, png, etc). Para ello es necesario especificar la **ruta al archivo (path)** que queremos acceder.

Como las rutas hacia los archivos, son *cadenas de caracteres* con un formato especial, necesitaremos usar el módulo `Path`, el cual esta contenido se encuentre contenido dentro del módulo `pathlib`

Veremos un ejemplo a continuación de cómo inportar un archivo `.py` que se encuentra "otro" directorio (no en la carpeta local).

Una *path* puede se definido llamando la función (o método) `Path(directorio)`, en donde, `directorio` es el string que contiene la ruta a la carpeta donde se encuentra el archivo que deseamos acceder.

```
In [ ]: 1 from pathlib import Path

In [ ]: 1 p=Path('~/Desktop') # Llamada "estilo" linux
        2
        3 q=Path('nombre_incorrecto**deDireccion') # También podemos usar otros métodos, algunos
        4                                           # y otros no, dependiendo de la variable de sy
```

Es posible añadir más información o extender el path luego.

```
In [ ]: 1 p=p / 'Introduction_To_Programming'
        2 print(p)
```

Los métodos `cwd` y `home`

Primero, podemos acceder el directorio donde se encuentra el archivo llamando al método `cwd` del `Path` (`'cwd'` significa, "directorio de trabajo actual", *'Current Working Directory'*).

```
In [ ]: 1 Path.cwd() # el método "cwd" retorna el Path de la localización del archivo en el cual

In [ ]: 1 Path.home() / 'Desktop/Ejemplo' # Será el nuevo directorio home o "base",
        2                               # desde el cual podemos definir la ruta a los archivos
        3                               # Usamos "/" par la concatenación de 'paths'
```

Dónde "apunta" el path?

A priori, un **path** debe ser una "ruta validad", es decir, si el string que pasamos como argumento no existe, tiene el formato incorrecto (o no tenemos acceso), puede ocurrir que no reporte un **Message Error**. Si queremos saber de ante-mano si el *path* es "valido", podemos utilizar el método `exists`.

```
In [ ]: 1 p=Path('path_inexistente') # Caulquier path "erroneo"
        2 print(p.exists()) # Usamos el método 'exists' para segurarnos que el path "existe" (o
        3 p=Path.home()/'Desktop' # Un path
        4 print(p.exists()) # Okay
```

También podemos chequear si un path apuntando a un directorio o archivo es correcto, utilizando los métodos `is_dir` y `is_file`

```
In [ ]: 1 p=Path.home() / 'UNaB' / 'StrucDatos' / 'Lectures' / 'Lecture9'
2
3 print("\nel path {} is apunta al directorio \" : {}".format(p,p.is_dir()))
4 print("\nel path {} is apunta al archivo \" : {}".format(p,p.is_file()))
5 p=p / 'Clase_9_Archivos.pdf' # Reemplazar por la ruta a un archivo existente
6
7 print("\nthe path {} is pointing to a directory \" : {}".format(p,p.is_dir()))
8 print("\nthe path {} is pointing to a file \" : {}".format(p,p.is_file()))
9
```

El método mkdir

Este método nos permite crear un directorio en un path (actual u otro path diferente).

```
In [ ]: 1 p=Path.home() / 'Desktop/Nowitexists'
2 print(p.exists())
3 p.mkdir() # Crea el directorio en el path asignado.
4 print(p.exists())
```

```
In [ ]: 1 p.mkdir() # Nos retornara error si el directorio ya existe...
```

```
In [ ]: 1 p.mkdir(exist_ok=True) # podemos suprimir algunos errores.
```

El método rmdir

Este método borra un directorio en un path especificado. **Como método de seguridad, sólo se pueden borrar directorios que estén vacíos.** Este método nos devuelve un error si el directorio no existe.

```
In [ ]: 1 p=Path.home() / 'Desktop/Nowitexists'
2 p.rmdir()
```

El método glob y parts

```
In [ ]: 1 # glob(pattern) retorna un iterador sobre una colección de todos los paths dado un "p"
2 # un parton siempre utiliza el carcter "*"
3
4 p=Path.home() / 'UNaB' / 'StrucDatos' / 'Lectures' / 'Lecture9'
5 list(p.glob('*.pdf'))
6
```

```
In [ ]: 1 # 'parts' es un atributo del path conteniendo todos el path "superior" (parents)
2
3 p.parts
4
```

Manipulación de Archivos

El módulo `os` nos permite mover archivos de un directorio a otro, también nos permite borrar archivos o directorios (entre otras funcionalidades).

Utilizamos el método `remove` del módulo `os` para borrar un archivo. Este método retorna un error si el archivo no existe en path dado.

```
In [ ]: 1 import os
```

```
In [ ]: 1 p=Path.home() / 'UNaB' / 'StrucDatos' / 'Lectures' / 'Lecture9'
2 os.remove(p / 'Clase_6_Alg_de_Busqueda_y_Ordenamiento.pdf')
```

Utilizamos el método `rename` para darle un nuevo nombre a un archivo o cambiar el directorio (similar a cómo `mv` es usado para renombrar o mover archivos en linux).

```
In [ ]: 1
        2 os.rename(p/'Clase_6_Alg_de_Busqueda_y_Ordenamiento.pdf', p/'Homework1000.pdf') # Camb
```

```
In [ ]: 1 os.rename(p/'Homework1000.pdf',p/'img'/'Lecture6.pdf')
        2 # Cambia el directorio donde se encuentra el archivo
```

Para copiar un archivo de un directorio a otro, debemos usar el módulo `shutil`.

Tenemos diferentes métodos para copiar archivos dentro de `shutil`: `copy`, `copy2`, y `copyfile`. En todos los casos debemos pasar los `path` del archivo y el directorio en donde queremos copiar.

Los métodos `copy` y `copy2` el `path` final del archivo a copiar, puede ser un `path` a un directorio existente o a un archivo "inexistente". En el primer caso, el archivo se copiará con su nombre original, en el segundo, se copiará con el nuevo nombre asignado.

Para el método `copyfile`, el segundo argumento no puede ser un directorio.

La diferencia entre `copy` y `copy2`, es que el segundo tratará de copiar todos los atributos de un archivo incluida la `metadata`. Cuando en duda, utilizar `copy2`.

```
In [ ]: 1 import shutil
```

```
In [ ]: 1 shutil.copy2(p / 'Clase_6.pdf',p / 'img' / 'Clase_6.pdf')
```

Apertura y cierre de Archivo

Con Python podemos precesar archivos de texto de manera automática. Para abrir un archivo utilizaremos el comando `open`.

Ejemplo: `open(path, string)`

El primer argumento es el `path` al archivo que queremos abrir. El segundo argumento, es opcional, y especifica que queremos hacer con el archivo, es decir, en que '*modo*' abriremos el archivo.

También podemos añadir '`b`' al segundo argumento para especificar que el archivo es '*binario*' y no un archivo de texto.

```
In [ ]: 1 p=Path.home() / 'UNaB' / 'StrucDatos' / 'Lectures' / 'Lecture9'
        2 f=open(p / 'clase7.py', 'r') # Crea una variable f que "contiene al archivo" (el 'handl
```

```
In [ ]: 1 print(type(f))
```

```
In [ ]: 1 print(f.read()) # Ejemplo
```

```
In [ ]: 1 f.close() # Cerrar el Archivo
        2 f.closed # Checquea si el archivo fue cerrado.
```

```
In [ ]: 1 f.read()
```

Cuando el archivo `f` es cerrado, la variable `f` continúa asignada al archivo, pero no podemos hacer nada con ella (sólo 're-abrirla'). Tratar de utilizar `f` generará un Error.

Dejar los archivos en los que hemos finalizado de trabajar por error, puede generar errores y decrementar la eficiencia de Python (debido al uso 'extremo' de memoria). Podemos utilizar el método `with` para asegurarnos que estamos trabajando con el archivo de manera apropiada.

```
In [ ]: 1 # Ejemplo:
2 with open(p/'clase7.py', 'r') as f:
3     print(f.read()) # Utilizar el archivo
4
5 f.closed # luego el archivo esta cerrado.
6         # Como podemos ver el archivo solo esta "abierto" dentro del bloque "with"
7
```

Archivos de Texto vs. Archivos Binarios

Cómo hemos mencionadom los archivo pueden ser 'archivos de texto', o 'archivos binarios'.

Ejemplos de *archivos de texto*:

- archivos de python .py
- archivos de texto .txt
- archivos LaTeX .tex
- archivos HTML .html
- ...

Cuando abrimos un archivo de texto, podremos ver inmediatamente que esta escrito en ella. Debemos tener en cuenta que la codificación de caracteres por defecto es **utf-8**.

Cuando tratamos de escribir en un archivo binario, como si fuese un archivo de texto, obtendremos un Error.

Importante : modificar un archivo binario es una operación mas compleja, ya que debemos saber como esta codificada la información.

```
In [ ]: 1 # Ejemplo
2 with open(p/'logistic_map.jpg', 'r+b') as f:
3     f.write(b'929838') # utilizamos b'.....' para utilizar un tipo especifico cuando
4
5     print(f.read(100))
```

Todos los archivos (incluso los rchivos de texto) estan codificados de forma binaria.

```
In [ ]: 1 with open(p/'dummy.txt', 'w') as f:
2     f.write(' A Frase a escribir') # Escribimos en el archivo
3
4 bytedata=bytes(b' A Frase a escribir') # Convertimos a un string binario
5 print(list(bytearray(bytedata))) # Vemos la informacion como caracteres desde el 0 has
6
7 with open(p/'dummy.txt', 'rb') as g: # Leer el archivo de forma binaria
8     databyte=g.read()
9
10 databyte==bytedata # ambos son iguales, los dos archivos de texto comparten la misma c
11
```

Commands para manipular Archivos

Cuando abrimos un archivo debemoms recordar que debemos especificar en que **modo**, es decir, **escritura**, **lectura**, **lectura/escritura**, o **agregar al final**.

Modo Lectura

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise.txt', 'r') # Abrimos el archivo en modo lectura 'r'.
```

El método `read()` retornara todo el contenido del archivo como un string de caracteres.

```
In [ ]: 1 textfile.read()
```

```
In [ ]: 1 textfile.read() # No contiene datos.
```

Cuando llamamos al método `read()` dos veces, la primer llamada retornara todo el texto, en cuanto la segunda retornara un string vacio. Esto es porque el método `read()` leera el archivo hasta el final (`EOF` , 'End Of File') y el no retorna el puntero al comienzo del archivo.

Para saber donde esta posicionado el '*puntero al archivo*', es decir, en que posicion estamos dentro del archivo, utilizaremos el método `tell` .

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise.txt','r')
2 print("Pointer before reading : ",textfile.tell())
3 textfile.read()
4 print("Pointer after reading : ",textfile.tell())
5 textfile.close()
```

Debemos ser cuidadosos al utilizar el método `tell` , especialemnte cuando utilizamos un indice.

```
In [ ]: 1 # Ejemplo:
2 textfile=open(p/'Adumbfile.txt','w')
3 textfile.write('Hola Mundo!')
4 textfile.close()
5 textfile=open(p/'Adumbfile.txt','r')
6 i=0
7 j=textfile.tell()
8 c=textfile.read(1)
9 while i<len('Hola Mundo!'):
10     j=textfile.tell()
11     c=textfile.read(1)
12     i+=1
13     print("character read : ",c,"| index ",i, "| tell value",j)
14
15
```

Tambien ocurre en el texto que estabamos utilizando.

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise.txt','r')
2 data=textfile.read(20000) # codifica los primeros 20 000 characters
3 textfile.close()
4 textfile=open(p/'A_madcap_Cruise.txt','r')
5 i=0
6 j=textfile.tell()
7 c=textfile.read(1)
8 while i<len(data):
9     i+=1
10     j=textfile.tell()
11     c=textfile.read(1)
12     print("character read : ",c,"| index ",i, "| tell value",j) # Ir a '19988'
13
14 # Existen algunos problemas con ciertos caracteres especiales....
15
```

El método `tell` funcionara correctamente sólo para '*archivos binarios*'. Debemos ser mas cuidadosos utilizando `tell` en archivos de texto.

Si queremos leer N caracteres, podemos agregar N como argumento al método `read` . También, podemos leer un archivo "línea a línea" utilizando el método `readline` (caracter de "corte de línea" `\n`).

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise.txt','r')
2 print(textfile.readline())
3 print(textfile.readline())
4 print(textfile.readline())
5 print(textfile.readline())
6 print(textfile.readline())
7 print(textfile.readline())
8 print(textfile.readline())
9 print(textfile.readline())
10 print(textfile.readline())
11 print(textfile.readline())
12 print(textfile.readline())
13 print(textfile.readline())
14 print(textfile.readline())
15 print(textfile.readline())
16 print(textfile.readline())
17 print(textfile.readline())
```

Finalmente, podemos notar que un archivo puede ser visto como un iterador. Será un iterador de las líneas contenidas en el archivo. Esta, es la mejor manera de recorrer/imprimir un archivo de texto.

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise.txt','r')
2 for x in textfile:
3     print(x)
4 textfile.close()
```

Modos de Escritura

Tenemos, a priori, tres modos de escribir sobre un archivo: 'w' , 'r+' , y 'a' . Los veremos en este orden a continuación.

```
In [ ]: 1 #p=Path.home()/'Desktop'
2 shutil.copy2(p/'A_madcap_Cruise.txt',p/'A_madcap_Cruise_copy.txt')
```

Si abrimos un archivo directamente en modo escritura 'w' , *todo será borrado en el archivo*

```
In [ ]: 1 with open(p/'A_madcap_Cruise_copy.txt','r') as f:
2     print(f.read(100))
```

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise_copy.txt','w')
2 # Ahora el archivo esta vacio?!?!
```

```
In [ ]: 1 textfile.close()
```

Importante: cuando unamos en modo 'w' para abrir un archivo, estamos sobre-escribiendo el archivo original y creando un archivo nuevo (en blanco) con el mismo nombre.

```
In [ ]: 1 p=Path.home()/'Desktop'
2 shutil.copy2(p/'A_madcap_Cruise.txt',p/'A_madcap_Cruise_copy.txt')
```

Para modificar una archivo utilizamos el modo 'r+' , el cual nos permite leer y escribir en un archivo.

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise_copy.txt','r+')
2 print(textfile.tell()) #The pointer is at the beginning
3 textfile.write('A sentence written somewhere you can see it!\n')
4 print(textfile.tell())
5 textfile.close()
```

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise_copy.txt','r')
2 print(textfile.readline())
3 print(textfile.readline())
4 textfile.close()
```

```
In [ ]: 1 # p=Path.home()/'Desktop'
        2 shutil.copy2(p/'A_madcap_Cruise.txt',p/'A_madcap_Cruise_copy.txt')
```

Para escribir al final de un archivo usamos el modo 'a' ('append' o "anadir").

```
In [ ]: 1 textfile=open(p/'A_madcap_Cruise_copy.txt','a')
        2 print(textfile.tell()) # El puntero esta al final
        3 textfile.write('Una frase, en un archivo!\n')
        4 print(textfile.tell()) # El puntero esta al final
        5 textfile.close()
```

```
In [ ]: 1 with open(p/'A_madcap_Cruise_copy.txt', 'r') as f:
        2     print(f.read(-100))
```

```
In [ ]: 1 with open(p /'clase.py', 'r') as f:
        2     print(f.read())
```